detectObject: inputs and outputs

Two USB webcams, one laptop. What goes in, what comes out, and why each was chosen. Companion to DETECT_OBJECT_DESIGN.md (black-box specification) and the code in detect_object/.

1. The box in one paragraph

detectObject receives a continuous stream of frames from two webcams mounted a known distance apart. For every pair of frames taken at nearly the same moment, it detects objects in each image, matches the two views into physical objects (so one chair seen by both cameras counts once and gets a distance), keeps only the objects inside a **focus region**, and then produces a ranked list of **facts** about that region. A single number, the **detail level λ ∈ [0, 1]**, decides how many of those facts are computed and reported. The output is one `DetectionReport` per frame pair, which is both human-readable (a summary sentence) and machine-readable (typed facts with rank, salience and confidence).

Stage	Box	Does	Code
Capture	S1.1	Pairs the newest left/right frames; skew ≤ 40 ms	capture.py
Geometry	S1.2	Camera poses, pixel rays, stereo overlap, effective focus region	geometry.py
Detector	S1.3	YOLO11n boxes in both images (OTS)	detector.py
Fusion	S1.4	Min-cost matching across cameras, triangulation	fusion.py
Focus filter	S1.5	Keeps objects inside the focus region	pipeline.py
Detail planner	S1.6	λ → what to compute (resolution, analyzers)	detail.py
Scene, lighting, hazards	S1.7–S1.9	Scene name, smoothed counts, light, trip hazards	scene.py, describe.py
Facts + selector	S1.10	Ranks candidate facts, selects by λ, writes the report	describe.py, detail.py

2. Inputs

There are four inputs. Only the first is a stream; the other three are configuration that changes rarely, so they are separate ports (rule R3 of the black-box format: configuration arrives on declared wires). The detector model is a fifth, install-time choice.

Input	Type and units	Default	Why this input
Camera streams (left, right)	BGR uint8 frames, 640×480 @ 30 fps each, MJPEG over USB 2.0; monotonic ns timestamps	devices 0 and 2 (Linux) / indices from --list (macOS)	Two viewpoints are what make distance measurable: a single camera gives bearing only. 640×480 MJPEG is the largest format two webcams can reliably share on one USB 2.0 bus, and it is more than the detector needs at λ ≤ 0.5.
Rig: baseline_m	metres, > 0	0.12	Sets the stereo depth resolution: $\delta Z \approx Z^2 \cdot \delta\theta / b$. Doubling b halves the distance error.
Rig: convergence_deg	degrees, (−90, 90); + = toed in	0	The angle between the cameras moves and shrinks the overlap region and bends every ray. Without it, triangulation is biased at every distance.
Rig: pitch_deg	degrees; + = looking down	0	Converts image rows into true elevation, so the vertical focus window and the floor plane are right.
Rig: height_m	metres above the floor, or None	None	Optional. Gives a floor plane, which turns 'is this a trip hazard?' into geometry (object ≤ 0.45 m tall, ≤ 4 m away) instead of a guess from image position.
Rig: per camera width_px, height_px, hfov_deg (or K, dist)	pixels; degrees; optional OpenCV 3×3 K + distortion	640, 480, 70°	The focal length in pixels converts pixels into angles. The datasheet HFOV is enough to start; a checkerboard calibration (K, dist) removes lens distortion when you need accuracy.

Input	Type and units	Default	Why this input
Focus region	yaw/pitch window (deg), near/far band (m), 2 flags	yaw ±30°, pitch ±25°, 0.3–10 m, clipped to stereo overlap	Says where to look, independently of the cameras. Defined in rig bearings, not pixels, so it stays valid when you change camera, resolution or mounting (section 3).
Detail level λ	float in [0, 1] or low/medium/high = 0/0.5/1	0.5 (medium)	One scalar is the simplest contract another algorithm can drive (e.g. raise λ when the user asks for more). It controls both cost and content (section 4).
Detector model (install time)	Ultralytics weights file	yolo11n.pt (COCO-80)	Fast on a laptop CPU, the same weights run on Linux and macOS. Use yolov8s-worldv2.pt with a vocabulary for classes COCO lacks (projector, speaker, whiteboard).

What I need from you. Measured baseline (lens centre to lens centre), the angle between the two cameras' optical axes, the common tilt, the mounting height if fixed, the webcam model (for HFOV), and which device is left. Put them in a JSON file (`python -m detect_object --write-config rig.json`) and pass `--config rig.json`.

3. The focus region

The focus region is a cone in rig bearings plus a distance band. An object is inside it when

```
yaw ∈ [yaw_min, yaw_max] ∧ pitch ∈ [pitch_min, pitch_max] ∧ (distance unknown ∨ distance ∈ [near, far])
```

Bearings are measured from the midpoint between the cameras: yaw positive to the right, pitch positive up, zero along the bisector of the two optical axes. With `clip_to_overlap` (default on) the window is intersected with the space both cameras see, and `near` is raised to where the two fields of view start to overlap.

Why this default

Choice	Reason (numbers for the default rig: b = 0.12 m, 70° HFOV, parallel)
Clip to the stereo overlap	Only where both cameras see can an object be triangulated and de-duplicated. For the default rig the overlap is yaw -35° to +35°, pitch -27.7° to +27.7°, starting 8.6 cm in front of the rig.
yaw ±30°, pitch ±25°	A few degrees inside the overlap on every side. Image edges have the most lens distortion and the most objects cut in half by the frame, which give wrong classes and wrong distances.
near 0.3 m	Closer than this, fixed-focus webcams blur and objects overflow the frame.
far 10 m	Stereo error grows with Z^2: about 2 cm at 1 m, 16 cm at 3 m, 46 cm at 5 m and 1.8 m at 10 m (1 px disparity error). Beyond 10 m a distance is no longer useful. Objects too far to triangulate are still kept, with distance = None.

Changing it

```
algo.set_focus(FocusRegion(yaw_min_deg=-10, yaw_max_deg=10, far_m=4))   # narrow, near
algo.set_focus(FocusRegion(require_stereo=True))   # only objects with a distance
# or in rig.json:  "focus": {"yaw_min_deg": -10, "yaw_max_deg": 10, ...}
```

4. The detail model

The detail parameter is a map from λ to two things: what is computed (a processing plan) and what is reported (a set of facts). Both are monotone in λ, and the reported set has a strict nesting property that downstream algorithms can rely on.

4.1 Candidate facts

Each frame pair produces a candidate set F. Every fact f has a **rank** r(f) ∈ [0, 1] (the smallest λ at which it may be reported), a **salience** s(f) ∈ [0, 1] and a **confidence** c(f) ∈ [0, 1].

```
scene r = 0
count of class c s_c = 0.35·min(1, ln(1+n)/ln 21) + 0.25·min(1, √A_c) + 0.40·I(c)
r = 0.05 + 0.70·(1 − s_c)
trip hazard r = 0.60
lighting r = 0.65
object location s_o = 0.5·√A_o + 0.25·I + 0.25/(1+d), r = 0.85 + 0.15·(1 − s_o)
```

n is the smoothed count, A the fraction of the image the class covers, I(c) a fixed importance table (person 1.0, screen 0.7, seat 0.6, default 0.3), d the distance. A class that is numerous, large or important gets a low rank and appears early; a single small, minor object appears only near λ = 0.75.

4.2 Selection

$$D(\lambda) = \text{first } N(\lambda) \text{ of sort}_{(r,\ -s,\ \text{subject})} \{ f \in F : r(f) \le \lambda \wedge (c(f) \ge 0.30 \vee f = \text{scene}) \}$$
$$N(\lambda) = \lfloor 1 + 39 \cdot \lambda \rfloor$$

Nesting theorem. If $\lambda_1 \le \lambda_2$ then $D(\lambda_1)$ is a prefix of $D(\lambda_2)$. Proof. The eligible set at λ_2 contains the one at λ_1; every fact added has rank $> \lambda_1$, so it sorts after all of them, because the sort key starts with r. Truncating to $N(\lambda_2) \ge N(\lambda_1)$ therefore keeps the first $N(\lambda_1)$ items, which are exactly $D(\lambda_1)$. ∎ In practice: a consumer can always cut a high-detail report down to a lower detail by keeping facts with rank ≤ λ, without re-running anything. This is tested on 200 random fact sets and end to end on the simulated room.

4.3 Processing plan

$$\text{imgsz}(\lambda) = 32 \cdot \text{round}((320 + 640 \cdot \lambda)/32) \rightarrow 320 \ldots 960 \text{ px detector input}$$
$$\text{run_hazards} \Leftrightarrow \lambda \ge 0.60$$
$$\text{run_lighting} \Leftrightarrow \lambda \ge 0.65$$
$$\text{run_locations} \Leftrightarrow \lambda \ge 0.85$$

A stage runs exactly when at least one fact it can produce is reportable, so nothing computed is thrown away. A larger detector input finds smaller and farther objects, which is what 'more detail' means physically. Measured on this laptop's CPU for two 640×480 frames: ≈ 60 ms at λ = 0, 180 ms at λ = 0.5, 380 ms at λ = 1. The nesting theorem holds for a fixed candidate set; since the plan changes the detector resolution, a higher λ can also find objects a lower λ missed.

4.4 What it looks like

λ	facts	imgsz	summary (simulated room: 60 seats, 36 of them inside the focus cone)
0.00	1	320	Conference room.
0.25	2	480	Conference room. 36 seats.
0.50	3	640	Conference room. 36 seats, one screen.
0.75	6	800	Conference room. 36 seats, one screen, one backpack. Possible trip hazard: backpack on the floor, ahead, 3.3 m. Lighting: normal, daylight-like light (about 8800 K).
1.00	11	960	Conference room. 36 seats, one screen, one backpack. Possible trip hazard: backpack on the floor, ahead, 3.3 m. Lighting: normal, daylight-like light (about 8800 K). Seat left, 2.3 m. Seat right, 2.3 m. Seat slightly right, 2.2 m. Screen ahead, 8.0 m. Backpack ahead, 3.3 m.

All ranks, the fact budget, the confidence floor and the resolution range are fields of `DetailModel`, so the model can be retuned without touching code.

5. Outputs

One `DetectionReport` per processed frame pair, as a Python object or as JSON (`report.to_dict()`, or `--json` on the command line, one line per report).

Field	Type	Why it is in the output
facts	list of Fact	The main output. Each fact is typed (kind, subject, value, unit) so a program can use it without parsing text, and carries rank, salience and confidence so the consumer can re-filter (nesting) or weigh it.
Fact.kind	scene | count | hazard | attribute | location	The level of abstraction, matching the three tiers in the brief: what kind of place, what is in it, and the details (light, hazards, where each thing is).
Fact.text	string	Ready to speak or print; also joined into summary.
Fact.object_ids	ints	Links a fact to the objects behind it, for highlighting or follow-up queries.
objects	list of SceneObject	Only the objects some selected fact refers to, so the payload grows with λ. Each has label, confidence, yaw/pitch, distance and 3D position (None when only one camera saw it), metric width/height, seen_by, and the box in each image.
scene, scene_confidence	string, 0–1	The λ = 0 answer, duplicated at the top level for consumers that need only that.

Field	Type	Why it is in the output
summary	string	Human-readable sentence(s), most important first.
plan	ProcessingPlan	What was actually computed (λ, detector size, which analyzers ran), so a consumer can tell 'not looked for' from 'not found'.
t_ns, latency_ms, skew_ms	ints / floats	Timing: when the frames were taken, how long processing took, how far apart the two frames were. Needed to fuse with other sensors.
warnings	list of strings	Degraded input is always stated, never silent: low scene confidence, camera skew > 40 ms, frame size not matching the rig.

Example (λ = 0.5, simulated room; object ids shortened)


```
{"detail": 0.5, "scene": "conference room", "scene_confidence": 0.61,
 "facts": [
  {"kind": "scene", "subject": "scene", "value": "conference room",
   "rank": 0.0, "salience": 1.0, "confidence": 0.61},
  {"kind": "count", "subject": "seat", "value": 36, "unit": "count",
   "rank": 0.20, "salience": 0.79, "confidence": 0.96,
   "text": "36 seats", "object_ids": [3, 4, 5, ...]},
  {"kind": "count", "subject": "screen", "value": 1, "unit": "count",
   "rank": 0.48, "salience": 0.39, "confidence": 0.99,
   "text": "one screen", "object_ids": [62]}],
 "summary": "Conference room. 36 seats, one screen.",
 "plan": {"detail": 0.5, "imgsz": 640, "run_lighting": false,
          "run_hazards": false, "run_locations": false},
 "latency_ms": 3.1, "skew_ms": 0.0, "warnings": []}
```


6. Limits to know about

Limit	Effect	Remedy
COCO-80 vocabulary	No 'projector' or 'speaker' class; screens and TVs are 'screen'.	YOLO-World weights + vocabulary (about 3× slower).
Rule-based scene names	10 categories from object counts; unusual rooms are 'unidentified scene'.	Swap S1.7 for a CLIP or VLM classifier (same signature).
Lighting from webcam colour	Auto white balance pulls the estimate toward daylight; confidence ≤ 0.5.	Lock white balance, or use a lux sensor.
No tracking across frames	Counts are per frame, median over 5 frames; a monocular object in the overlap may be counted twice.	Add a tracker between S1.5 and S1.7.
Trip hazards	Only detected low objects, not holes, cables or steps.	Set height_m; add a depth-based floor check.
Unsynchronised cameras	Up to 40 ms between views; fast motion can break matches.	Cameras with a hardware sync input.

Linux and macOS. The only platform-specific line chooses the capture backend (V4L2 on Linux, AVFoundation on macOS). Cameras are opened on the main thread and the optional preview window is drawn on the main thread, as macOS requires. The detector uses CUDA, Apple MPS or the CPU, whichever is available. On macOS, give the terminal camera permission and find the indices with `python -m detect_object --list`.